Peat Fire Early-Warning Agent: Team Playbook

IBM Z Datathon 2026 | 24h build | 5 people, 5 AI agents

Give this whole file to every agent. Each person owns ONE role. Do not edit another role's folder. Do not change a contract without announcing it and bumping `schema_version`.

---

0. Global rules

1. **Contracts are law.** Sections 2 and 3 define the exact JSON/column names. Build against them, not against your assumptions.
2. **Mocks first.** Within hour 2, every role commits a `mock_output.json` that is valid against the contract. Downstream roles use it until real output exists.
3. **All timestamps are UTC**, ISO-8601 with `Z` (e.g. `2026-10-17T09:00:00Z`).
4. **Coordinates are `lat, lon` in JSON fields**, but GeoJSON arrays are `[lon, lat]`. This is the most likely bug in the project. Convert once, in Role A, and nowhere else.
5. **Repo layout**

```
/contracts        CONTRACTS.md + example files (frozen)
/a_data           Role A
/b_model          Role B
/c_agent          Role C
/d_frontend       Role D
/e_platform       Role E (Dockerfiles, deploy scripts, screenshots, video)
/data/raw         downloaded source files (gitignored if large)
/data/processed   features.json, risk.json, dispatch.json
```

6. **Code must be written during Oct 17-18.** Before that: research, abstract, architecture, reading docs only.
7. Every open-source library and dataset used goes into `README.md` with its link (submission requirement).
8. Feature freeze at hour 18. After that: bugs, demo, video only.

---

1. Source data formats (verified from the real files)

1.1 Open-Meteo hourly (CSV)

The CSV is **two blocks in one file**: a 1-row metadata block, a blank line, then the hourly block.

```
latitude,longitude,elevation,utc_offset_seconds,timezone,timezone_abbreviation
<one row per location>

time,temperature_2m (°C),precipitation (mm),wind_speed_10m (km/h),wind_direction_10m
(°),
soil_moisture_0_to_1cm (m³/m³),soil_moisture_1_to_3cm (m³/m³),soil_moisture_3_to_9cm
(m³/m³),
soil_moisture_9_to_27cm (m³/m³),soil_moisture_27_to_81cm
(m³/m³),vapour_pressure_deficit (kPa),
soil_temperature_0cm (°C),soil_temperature_18cm (°C),evapotranspiration
(mm),et0_fao_evapotranspiration (mm)
2026-09-28T00:00,14.7,0.00,11.4,125,0.281,...
```

Gotchas:

- **Encoding is corrupted in headers** (`Â°C`, `mÂ³/mÂ³`). Never match full header strings. Use `col.split(" (")[0]` to get the clean name (`temperature_2m`).
- Metadata and data are separate blocks, so a naive `pd.read_csv` breaks. For live pulls, **use `format=json` on the API instead**. Use the CSV only as cached fixtures.
- `time` has no timezone suffix. It is local to the `timezone` param. **Always request `timezone=GMT`** so it is UTC. Append `Z` in code.
- One metadata row = one location. Cache per location.
- There is **no humidity column**. Use `vapour_pressure_deficit` as the dryness signal.
- The sample (Berlin, 52.52/13.42) is a format example only. Real requests use Indonesian coordinates.

Canonical rename map (use exactly these names everywhere downstream):

Open-Meteo column	Canonical name
temperature_2m	`temp_c`
precipitation	`precip_mm`
wind_speed_10m	`wind_kmh`
wind_direction_10m	`wind_dir_deg`
soil_moisture_0_to_1cm	`sm_0_1`
soil_moisture_1_to_3cm	`sm_1_3`
soil_moisture_3_to_9cm	`sm_3_9`
soil_moisture_9_to_27cm	`sm_9_27`
soil_moisture_27_to_81cm	`sm_27_81`
vapour_pressure_deficit	`vpd_kpa`
soil_temperature_0cm	`soil_temp_0_c`
soil_temperature_18cm	`soil_temp_18_c`
evapotranspiration	`et_mm`
et0_fao_evapotranspiration	`et0_mm`

1.2 Peatland GeoJSON (Global Forest Watch, Indonesia)

- `FeatureCollection`, CRS is **CRS84 =** `[lon, lat]` **order**.
- Geometry is `MultiPolygon`, coordinates are **3D** `[lon, lat, 0.0]`. Drop the third value.
- Properties: `objectid_1`, `objectid`, `layer_revi` ("gambut" = peat), `shape_leng`, `shape_Length`, `globalid` (use as the unique polygon id), `shape_Length`, `shape_Area`.
- `shape_Area` looks like square meters (e.g. 32,863,012 = about 32.9 km²). Verify by recomputing area with a projected CRS (EPSG:32748 or 32647 depending on region) before trusting it.
- **There is no peat depth, no peat type, no province name** in the properties. Do not promise depth-based features. Province can be derived with a spatial join if needed.
- The file is large. Filter to the demo province bounding box first, simplify geometry (`simplify(0.001)`), and build a spatial index (`shapely.STRtree` / `geopandas.sindex`).

1.3 NASA FIRMS (VIIRS NRT, CSV)

```
latitude,longitude,bright_ti4,scan,track,acq_date,acq_time,satellite,instrument,confide
```

- `acq_date` = `YYYY-MM-DD`, `acq_time` = **HHMM as an integer without padding, UTC** (`1` means 00:01, `930` means 09:30). Zero-pad to 4 digits before parsing.
- `satellite`: single letter code (N / 1 / 2 = different VIIRS satellites). Keep it, do not interpret it.
- `confidence` for VIIRS is a letter: `l` low, `n` nominal, `h` high. Drop `l` for alerts, keep all for training with the level as a feature.
- `frp` = fire radiative power (MW). Useful as intensity.
- `daynight`: `D` / `N`.
- The sample row (2.9, 36.1) is not in Indonesia. Always filter to the Indonesia/demo bounding box and then to hotspots near or inside peat polygons.
- Real downloads need a FIRMS MAP_KEY. Role A owns it.

---

2. Grid and IDs

- The unit of prediction is a **grid cell**: a 0.1° x 0.1° square (about 11 km, matching the underlying resolution of the soil data) that intersects peat.
- `cell_id = "{lat_center:.2f}_{lon_center:.2f}"`, for example `"-2.35_114.05"`. Cell centers are on `.x5` values.
- Only cells with `peat_fraction >= 0.05` exist in the system.
- **Demo region:** pick ONE province (suggestion: Central Kalimantan or Riau) in hour 0, and tell everyone. Cap at about 300 cells so API calls and demo both stay fast.

---

3. Contracts

3.1 `features.json` (A -> B). One object per cell per hour.

```json
```

```
{
  "schema_version": "1.0",
  "cell_id": "-2.35_114.05",
  "ts": "2026-10-17T09:00:00Z",
  "lat": -2.35,
  "lon": 114.05,
  "peat": { "peat_fraction": 0.62, "peat_area_km2": 7.4, "polygon_ids": ["{DF27C432-..
  "weather": {
    "temp_c": 33.1, "precip_mm": 0.0, "wind_kmh": 11.4, "wind_dir_deg": 125,
    "sm_0_1": 0.18, "sm_1_3": 0.19, "sm_3_9": 0.21, "sm_9_27": 0.24, "sm_27_81": 0.28,
    "vpd_kpa": 2.1, "soil_temp_0_c": 36.0, "soil_temp_18_c": 31.4,
    "et_mm": 0.3, "et0_mm": 0.4
  },
  "derived": {
    "rain_24h_mm": 0.0, "rain_7d_mm": 3.2, "rain_30d_mm": 41.0,
    "days_since_rain": 9,
    "sm_27_81_delta_7d": -0.04,
    "vpd_mean_24h": 1.9,
    "soil_air_temp_gap_c": -1.7
  },
  "fire": {
    "hotspots_in_cell_24h": 0,
    "hotspots_10km_7d": 4,
    "frp_sum_10km_7d": 18.3,
    "nearest_hotspot_km": 6.2
  }
}
```


Rules: `days_since_rain` counts days since the last day with at least 1 mm of rain. Missing values are `null`, never `0` or `-999`. Deliver as a JSON array in `/data/processed/features.json`, plus `features.parquet` for training.

3.2 `risk.json` (B -> C, D)

json

```
{
  "schema_version": "1.0",
  "cell_id": "-2.35_114.05",
  "ts": "2026-10-17T09:00:00Z",
  "horizon_h": 48,
  "risk_score": 0.87,
  "risk_level": "CRITICAL",
  "top_factors": [
    { "feature": "days_since_rain", "contribution": 0.31 },
    { "feature": "sm_27_81", "contribution": 0.22 }
  ],
  "model_version": "v1"
}
```

`risk_level` thresholds (fixed, shared): `LOW < 0.35 <= MODERATE < 0.60 <= HIGH < 0.80 <= CRITICAL`. `horizon_h` is 24, 48 or 72.

3.3 `dispatch.json` (C -> D, E)

json

```
{
  "schema_version": "1.0",
  "dispatch_id": "d_20261017_0001",
  "cell_id": "-2.35_114.05",
  "ts": "2026-10-17T09:05:00Z",
  "action": "DISPATCH",
  "priority": 1,
  "target": { "lat": -2.35, "lon": 114.05 },
  "audiences": ["manggala_agni", "mpa", "bpbd"],
  "messages": {
    "manggala_agni": { "id": "…", "en": "…" },
    "mpa": { "id": "…", "en": "…" },
    "bpbd": { "id": "…", "en": "…" },
    "brgm": { "id": "…", "en": "…" },
    "leadership": { "id": "…", "en": "…" }
  },
  "water_source": { "lat": -2.33, "lon": 114.02, "distance_km": 3.1, "kind": "canal" },
  "rationale": "Risk 0.87 for 48h; 9 days without rain; deep soil moisture falling.",
  "status": "PENDING"
}
```

`action` is one of `DISPATCH`, `SMS_WARN`, `REWET`, `MONITOR`, `BRIEF`. `priority` is 1 (highest) to 4. `water_source` may be `null`. The `messages` object only needs keys for the listed audiences. Write messages in Indonesian (`id`) and English (`en`).

---

4. Roles

Role A: Data engineer

Mission: turn the three raw sources into `features.json`. Everyone depends on you first, so ship a mock in hour 1-2 and a real small sample by hour 4.

Inputs: peat GeoJSON, Open-Meteo (API, `format=json`, `timezone=GMT`, `past_days=30`, `forecast_days=3`), FIRMS CSV. **Outputs:** `/data/processed/features.json`, `features.parquet`, `cells.geojson` (cell polygons with `cell_id` and `peat_fraction` for the map), and a historical training set `history.parquet` (same schema, 1+ year, with a label column from B's spec).

Tasks:

1. Filter the peat GeoJSON to the demo province, drop the Z coordinate, simplify, build the spatial index.
2. Generate 0.1° cells that intersect peat. Compute `peat_fraction`, `peat_area_km2`, `polygon_ids`.
3. Pull Open-Meteo per cell center (batch comma-separated coordinates). Cache raw responses in `/data/raw`. Apply the rename map in section 1.1.
4. Load FIRMS, parse `acq_date` + zero-padded `acq_time` to UTC, drop `confidence == "l"` for the alert feed, spatial-join to cells.
5. Compute all `derived` and `fire` fields.
6. Provide `refresh.py` that rebuilds the latest features in under 2 minutes.
7. Build the historical dataset for B (Open-Meteo archive API plus FIRMS archive).

Done when: validates against the schema, no cell has all-null weather, a unit test checks the lon/lat order.

Role B: ML lead

Mission: predict the probability of a hotspot emerging within a peat cell in the next 24/48/72h, from `features.json`.

Inputs: `history.parquet` (training), `features.json` (live). **Outputs:** `/data/processed/risk.json`, `model/` artifact, `MODEL_CARD.md`, `predict.py` (reads features, writes risk).

Tasks:

1. Label: `1` if at least one FIRMS hotspot (confidence n/h) falls in the cell within the next `horizon_h`, else `0`. Handle heavy class imbalance. Use a **time-based** train/validation

split, never random.
2. Baseline: logistic regression plus a rule-based dryness index. Then gradient boosting.
3. Use IBM-optimized libraries where possible (for example Snap ML on LinuxONE) and show it, since it is judged under "Technical Design". Coordinate with Role E.
4. Output top contributing factors (SHAP or feature importances) per prediction in `top_factors`.
5. Report honestly: precision/recall, lead time, and the limitation that FIRMS detects **surface** hotspots, so the model predicts surface emergence from subsurface-dryness conditions rather than directly observing underground smoldering.
6. Soil data is modeled, not sensor telemetry. State that plainly and show how real sensor readings would drop into the same `weather` fields.

Done when: `predict.py` runs end-to-end on A's real features in under 1 minute and the output validates against 3.2.

Role C: Agent engineer

Mission: turn risk into actions. Decide who is notified, with what message, to what location.

Inputs: `risk.json`, `features.json`, `cells.geojson`. **Outputs:** `/data/processed/dispatch.json`, `agent.py` (loop: read risk, decide, write dispatch), `decision_rules.md`.

Tasks:

1. Deterministic policy first, LLM second. Example: `CRITICAL` -> `DISPATCH` to Manggala Agni + BPBD, `HIGH` plus falling `sm_27_81` -> `SMS_WARN` to MPA and `REWET` to BRGM, any CRITICAL cluster -> `BRIEF` to leadership.
2. Dedupe and cool-down: one active dispatch per cell, escalate only if the level rises.
3. Cluster adjacent CRITICAL cells into one incident with a single target coordinate.
4. Use an LLM only to write the Indonesian/English messages and rationale from structured fields. Never let it invent coordinates or numbers. Validate output against 3.3.
5. Water source routing: **none of our three datasets contains water sources.** Decide in hour 0 between (a) OpenStreetMap canals/rivers via Overpass for the demo province, or (b) a small hand-made list of canals for the demo cells. If neither is ready, set `water_source: null` and say so.
6. Alerts are **simulated** (written to a log and shown in the UI). No real SMS is sent.

Done when: feeding `risk.json` with a CRITICAL cell produces a valid dispatch with all five audience messages.

Role D: Frontend / alerts

Mission: the visible product. A command-center dashboard that makes the 3-minute demo easy.

Inputs: `cells.geojson`, `risk.json`, `dispatch.json` (read via simple HTTP from Role E's API or static files). **Outputs:** `d_frontend/` web app.

Tasks:

1. Map: peat polygons or cells colored by `risk_level` (LOW green, MODERATE yellow, HIGH orange, CRITICAL red), tooltip with `top_factors`.
2. Role views (a switcher is enough): Manggala Agni (target coordinate, water source, route), MPA (SMS preview), BPBD (heatmap plus active dispatches), BRGM (soil moisture trend), Leadership (summary counts and a one-line brief).
3. Time slider for 24/48/72h horizons.
4. A "simulate surge" button that injects a dry-spell scenario so the demo shows a dispatch happening live.
5. Works from static mock files with no backend, so it never blocks the demo.

Done when: the full demo flow runs offline from mock JSON.

Role E: Platform, integration and pitch

Mission: make it run on IBM Z / LinuxONE, glue the pipeline, and own the submission. This role carries the Technical Design score and the Execution score.

Inputs: everyone's code. **Outputs:** deployed system, `docker-compose.yml`, `scripts/run_pipeline.sh`, screenshots, flowchart, 3-minute video, README, submission.

Tasks:

1. **Before Oct 17:** finish both mandatory IBM labs and attend the Oct 16 setup session. Confirm access to the LinuxONE instance and Jupyter. Share the access steps with the team.
2. Build `run_pipeline.sh`: A refresh -> B predict -> C agent -> files served to D. Containerize each stage (Dockerfile per role).
3. Verify every dependency has an s390x build or installs on LinuxONE. Find this out in hour 1-2, not hour 20.
4. Run the model and agent **on LinuxONE**, and capture screenshots as evidence: instance dashboard, containers running, model inference, resource usage.
5. Maintain the contracts folder and run a schema validation script in CI or as a pre-merge check.
6. Draw the architecture flowchart (optional submission item, high value).
7. Video (max 3 min): problem (20s), data (20s), solution and live demo (80s), **how it uses IBM Z / LinuxONE: tech stack and the running instance (40s)**, impact and

audiences (20s). Rehearse once.

8. Submit through Hack2Skill with title format `[TeamName_IBM Z Datathon 2026 Project]`, GitHub link, video, and open-source links. Submit with at least 30 minutes of buffer.

---

5. Hour-by-hour

Hours	Everyone
0-1	Confirm demo province, freeze contracts, create repo, set up folders, E confirms LinuxONE access
1-2	Every role commits `mock_output.json`
2-8	Build in isolation against mocks
8-12	First end-to-end run on real data (A -> B -> C -> D)
12-18	Improve model and agent, deploy on LinuxONE, D finishes role views
18	**Feature freeze**
18-21	Bug fixes, screenshots, flowchart
21-23	Record and edit video, finish README
23-24	Submit with buffer

6. Agent prompt header (paste at the top of each agent's session)

```
You are the agent for Role <X> in a 5-person hackathon team. Read TEAM_PLAYBOOK.md
fully.
Only edit files in your own role folder. Your inputs and outputs must match
the contracts in section 3 exactly (field names, units, UTC timestamps).
Never rename fields. If you think a contract must change, stop and tell the
human instead of changing it. Commit a valid mock_output.json first.
GeoJSON is [lon, lat]; JSON fields are lat, lon. Use null for missing values.
```

7. Known risks to decide early

- No real soil sensor feed: soil data is modeled (Open-Meteo). State this openly.
- FIRMS only sees surface hotspots: frame the model as early prediction of surface emergence.
- No peat depth in the GeoJSON: do not use depth features.
- No water-source data: decide OSM vs hand-made list in hour 0 (Role C).

- Open-Meteo API rate limits: cache aggressively, small demo region.
- s390x compatibility of Python packages: Role E checks in the first two hours.